UNITED STATES PATENT AND TRADEMARK OFFICE

PATENT APPLICATION

System and Method for Deterministic Platform-Enforced Orchestration of Stateless AI Agents Across Multi-Phase Expert Workflows with Outcome-Driven Adaptation

NON-PROVISIONAL APPLICATION	No. 19/646,571
PROVISIONAL APPLICATION	No. 64/015,378
PRIORITY DATE	March 24, 2026
FILED	April 14, 2026
INVENTOR	Sanjay Sahgal
ENTITY STATUS	Small Entity
TOTAL CLAIMS	20 (3 independent, 17 dependent)
TECHNICAL DRAWINGS	17
STATUS	Patent Pending

SECTION I

Abstract

A platform for orchestrating stateless artificial intelligence agents across multi-phase expert workflows, using version control as the sole authoritative state store. The platform deterministically enforces agent behavior compliance through structural gates that operate independently of prompt instructions, ensuring that required behaviors occur reliably regardless of whether the underlying language model follows its instructions. The system eliminates the need for persistent agent processes, external workflow databases, or manual context relay between domain experts.

SECTION II

Background

Existing tools for software development, project management, and business operations fall into two categories: single-agent assistants that operate within one domain (coding, design, project tracking), and multi-agent frameworks that provide orchestration primitives without domain-specific agents or behavioral enforcement.

Neither category provides a unified platform in which domain-expert agents coordinate across the full product lifecycle, maintain consistency through deterministic verification, and adapt their behavior based on production outcomes. The human operator remains responsible for relaying context between tools, validating cross-phase consistency manually, and translating production metrics into updated instructions.

This patent application addresses **eleven specific limitations** present across all existing systems:

1	**Stateful architecture**	Agents require persistent processes or databases; crash recovery requires state reconstruction.
2	**Hardcoded specificity**	New teams require duplication and modification of configuration; no abstraction layer exists.
3	**Post-hoc consistency**	Inconsistencies are discovered after propagation downstream, not prevented at creation time.
4	**Truncation-based context**	Documents exceeding context windows are truncated rather than filtered for relevance.
5	**Separate agents per mode**	Different agent instances for different states; routing complexity grows multiplicatively.
6	**Duplicated CI/CD logic**	Every repository maintains its own pipeline configuration; drift is inevitable.
7	**Unstructured specifications**	Open questions exist as free-text comments with no automated routing or gating.
8	**Opaque workflow state**	Stakeholders query multiple systems to understand project status.
9	**Prompt-dependent governance**	Agent behavior governed by natural language instructions; compliance is probabilistic.
10	**Open-loop execution**	No feedback from production outcomes to agent behavior; humans relay everything.

11	**No unified platform**	No system orchestrates expert agents across all business functions holistically.

SECTION III

Prior Art

Nineteen existing systems were analyzed against the invention's eight required capabilities. No system provides all eight. The following table summarizes the most relevant prior art and the specific capabilities each lacks.

SYSTEM	WHAT IT DOES	WHAT IT LACKS
GitHub Copilot Workspace	Single generalist agent. Takes issues, produces pull requests. Relies on existing CI/CD and human review.	No lifecycle phases. No spec validation. No enforcement gates.
Devin (Cognition)	Autonomous software engineering agent. Most well-funded coding agent as of filing date.	Single agent. No PM, designer, or architect. No internal enforcement.
MetaGPT	Simulates a software company with PM, Architect, Engineer, QA roles. Closest SDLC analog.	Prompt-based verification only. No multi-tenant. No production feedback loop.
LangChain / AutoGen	Agent orchestration frameworks providing infrastructure primitives.	No domain agents. No structural verification. No multi-tenant architecture.
CrewAI	Multi-agent framework with hierarchical coordination and role-based agents.	No structural output verification. No enforcement gates. No self-extension.
Factory AI	Task-specific coding agents. DroidShield performs structural enforcement for code security.	No product-domain compliance. No closed-loop outcome adaptation.
Metaswarm	Multi-agent framework with mandatory quality gates at every handoff. Closest enforcement analog.	Engineering-only enforcement. No PM or design agents. No multi-tenant.
Soleur	63 agents across 8 business departments. Closest full-company analog.	No deterministic enforcement. No version-control state. No escalation protocol.
ChatDev	Virtual software company with chat-chain waterfall process.	Prompt-based only. One-shot generation. No persistence or feedback.
Replit Agent 3	Autonomous app builder. Self-tests code. Can generate sub-agents.	Single generalist. Proprietary environment. No domain expert personas.

SECTION III (CONTINUED)

Prior Art

SYSTEM	WHAT IT DOES	WHAT IT LACKS
Linear / Jira + Rovo	Workflow state in proprietary databases. Rovo Dev performs automated code review.	State outside version control. No PM or design agents natively.
Microsoft Agent Governance Toolkit	Deterministic policy enforcement: trust gates, reliability gates, sub-millisecond governance.	Enforces security and safety only — not product-domain compliance.
Cursor / Windsurf / vo	Single-agent coding assistants or application builders.	No multi-agent orchestration. No domain specialization. No enforcement.
Amazon Q / OpenHands	Autonomous coding agents with IDE or terminal integration.	Engineering-only. No cross-phase orchestration.

Eight Required Capabilities

The invention is distinguished by the simultaneous presence of all eight capabilities. No prior art system provides more than three.

a	**Version control as sole state**	All workflow state, specifications, and phase transitions stored in and derived from version control.
b	**Stateless agents**	Pure-function invocations with relevance-filtered context from a secondary model.
c	**Synchronous conflict detection**	Upstream validation with re-read verification before artifact save.
d	**Phase-aware behavior switching**	Single agent instance adapts via system prompt injection; no separate processes.
e	**Zero-code-change multi-tenancy**	New team onboards with environment variables only. No codebase modifications.
f	**Version-control dashboard**	Real-time monitoring derived entirely from Git state with no separate data store.
g	**Deterministic enforcement**	Structural gates verify agent compliance independently of prompt instructions.
h	**Closed-loop outcome adaptation**	Production metrics feed back into agent behavior without human relay.

SECTION IV

Description of the Invention

4.1 System Architecture

The system comprises three layers: a **human interaction layer** (decoupled via event protocol, supporting Slack, web, CLI, or any future interface), a **platform orchestration layer** (routing, phase detection, enforcement, context injection), and a **version control layer** (the sole authoritative state store for all workflow state, specifications, and decisions).

4.2 Stateless Agent Invocation

Each agent is a stateless function. On every invocation, the platform: (1) reads Git branch and file state to determine lifecycle phase; (2) loads all upstream specifications from version control; (3) filters loaded documents for relevance using a secondary language model; (4) constructs an invocation payload comprising system prompt, filtered context, and conversation history; (5) invokes the domain-expert agent; and (6) validates the agent's output through five independent enforcement layers before committing any artifacts.

4.3 Deterministic Platform Enforcement

The invention's central contribution is a five-layer enforcement architecture that operates independently of whether the language model follows its prompt instructions:

Layer 1 — Pre-run structural gates. Before the agent is invoked, the platform checks preconditions (e.g., blocking questions must be resolved before finalization). If preconditions fail, the agent is never called.

Layer 2 — Post-run output verification. After the agent responds, the platform parses the response to verify that required output elements are present. Missing elements are corrected by re-running the specific failing component.

Layer 3 — State-based finalization gates. Before any specification is committed to version control, the platform runs a structural audit. The audit counts required format markers rather than pattern-matching against undesirable text — verifying presence of correct output rather than absence of incorrect output.

Layer 4 — Always-on proactive audits. On every message, domain-specific audits run automatically: conflict detection against upstream specifications, brand token drift analysis, and phase completion readiness. Results are injected into agent context as platform notices.

Layer 5 — Fallback classification. If none of the structural gates fire, a secondary language model classifies the agent's response to determine whether additional platform action is required.

4.4 Specification Chain

Specifications form a directed chain: Product Specification → Design Specification → Engineering Specification → Build. Each downstream agent reads the full upstream chain from version control. The conflict detection system validates every specification against all upstream documents before save, blocking contradictions and surfacing gaps in real time.

Description of the Invention

4.5 Cross-Phase Escalation Protocol

When a downstream agent (e.g., Design) detects a blocking question owned by an upstream phase (e.g., Product), it invokes the upstream agent directly within the current conversation thread. The human operator never relays context manually between agents. Multi-layer gates ensure escalation fires even if the agent fails to call the escalation tool — the platform detects the unresolved question and triggers escalation deterministically.

4.6 Multi-Tenant Architecture

Every product-specific coordinate — repository path, channel names, specification file paths, product name — is externalized into a workspace configuration object loaded from environment variables. No product-specific string literal appears in the codebase. A new team onboards by providing a single environment configuration file. The platform codebase requires zero modifications. Continuous integration enforces this constraint on every commit.

4.7 Outcome-Driven Adaptation

The system closes the loop between production outcomes and agent behavior. Production metrics (user engagement, error rates, performance data) are ingested and used to modify agent context for subsequent invocations. Agents whose previous specifications led to measurable production issues receive updated context reflecting those outcomes, enabling self-correcting behavior without human intervention.

4.8 Cross-Domain Specification Chain Composition

The specification chain architecture generalizes beyond software development to any domain requiring coordinated multi-phase expert workflows. Multiple linear chains (e.g., SDLC chain, GTM chain, finance chain) compose via typed handoff interfaces, enabling the platform to orchestrate agents across software development, marketing, sales strategy, financial modeling, and go-to-market planning.

4.9 Autonomous Agent Generation

The platform can generate new agent configurations — including persona definitions, rubric criteria, enforcement gate specifications, and specification chain position — without code changes. When a new domain expert is needed, the platform scaffolds the complete agent configuration. Generated configurations inherit the full enforcement architecture automatically.

SECTION V

Embodiments

The invention is described through seventeen preferred embodiments, each addressing a distinct inventive mechanism. Together they form the complete platform architecture.

1	**Version Control as Workflow State Machine**	Lifecycle phase inferred from Git branch and file state. No separate workflow database.
2	**Stateless Agent Architecture**	Agents are pure functions with fresh context per invocation. Prompt caching and relevance filtering via secondary model.
3	**Synchronous Conflict Detection**	Specifications validated against upstream documents before save. Re-read verification prevents human circumvention.
4	**Phase-Aware Behavior Switching**	Single agent instance switches behavioral modes via system prompt injection based on detected phase.
5	**Config-Driven Multi-Tenancy**	All tenant-specific coordinates externalized to environment configuration. New team onboarding requires zero code changes.
6	**Decoupled CI/CD Platform**	Pipeline logic centralized. Applications onboard via single workflow file. Improvements auto-inherited.
7	**Structured Open Question Tagging**	Open questions tagged with type and blocking metadata for automated routing and phase gating.
8	**Interface-Agnostic Interaction Layer**	Human interaction decoupled via event protocol. Interface substitution requires zero orchestration changes.
9	**Version-Control Dashboard**	Real-time project monitoring derived entirely from Git state with no separate data store.
10	**Specification Chain with Conflict Enforcement**	Linear chain with full upstream injection and synchronous contradiction blocking.
11	**Cross-Phase Agent Escalation**	Downstream agents invoke upstream agents directly in the conversation thread.
12	**Proactive Phase Orchestration**	Monitors Git state across features. Gates phase transitions. Detects workflow stalls.
13	**Deterministic Multi-Layer Enforcement Gates**	Five-layer verification architecture operating independently of prompt compliance.
14	**Outcome-Driven Behavioral Adaptation**	Production metrics close the loop into agent behavior for subsequent invocations.
15	**Cross-Domain Specification Chain Composition**	Multiple linear chains compose via typed handoff interfaces across business domains.
16	**Autonomous Agent Configuration Generation**	Platform generates new agent personas, rubrics, and enforcement rules without code changes.
17	**Multi-Layer Proactive Audit Architecture**	Entry-time, save-time, and completion-gate audits execute automatically on every message.

SECTION VI

Claims Summary

The application contains 20 claims: 3 independent claims establishing the invention across system, method, and computer-readable medium categories, and 17 dependent claims specifying detailed mechanisms.

INDEPENDENT CLAIMS

1 **System.** A system for orchestrating stateless AI agents across multi-phase expert workflows comprising: a version control layer as sole authoritative state store; a phase detection module that infers lifecycle phase from version control state; a context injection module that constructs invocation payloads with relevance-filtered upstream specifications; a stateless agent invocation module; and a platform enforcement module that verifies agent output compliance through structural gates independent of prompt instructions.

12 **Method.** A method for orchestrating stateless AI agents comprising the steps of: inferring a current lifecycle phase from version control state; constructing a context payload from upstream specifications filtered by a secondary language model; invoking a domain-expert agent as a stateless function; and returning the agent response after platform enforcement verification.

18 **Computer-readable medium.** A non-transitory computer-readable medium storing instructions for orchestrating stateless AI agents, the instructions implementing the system of Claim 1 including deterministic enforcement gates, cross-phase escalation, and proactive audit architecture.

DEPENDENT CLAIMS (SUMMARY)

2–3 Synchronous conflict detection with re-read verification; config-driven multi-tenant architecture with CI enforcement.

4–6 Five-layer deterministic enforcement gates; outcome-driven behavioral adaptation from production metrics; autonomous agent configuration generation.

7–11 Phase transitions via Git operations; invocation payload structure with prompt caching; phase-aware behavior switching; structured question tagging; specification chain with conflict enforcement.

13–17 Method claims: deterministic enforcement in method form; cross-phase escalation protocol; multi-tenant method; proactive orchestration method; outcome metrics ingestion method.

19–20 Medium claims: multi-tenant zero-code-change onboarding; proactive multi-layer audit orchestration.

FILING DETAILS

APPLICATION TYPE	Utility — Nonprovisional under 35 U.S.C. 111(a)
CLASSIFICATION	Artificial Intelligence; Software Development Tools; Multi-Agent Systems
SPECIFICATION	28 pages
TECHNICAL DRAWINGS	17 figures
PRIOR ART ANALYZED	19 systems
APPLICABLE DOMAINS	Software development, product management, design, engineering, finance, marketing, sales, go-to-market planning, and autonomous generation of new agent configurations